3. If a change is declared, then (a) the point p_c at time t-*changeResetWindow* is marked as a change point, (b) all points after it are unmarked (they are not outliers anymore), and (c) the model estimation is started afresh from p_c.

Table 1. Parameters for deciding outliers/changes. Parameters with * are user–provided.

Parameter	Definition
*minDetectionWindow**	at least *minDetectionWindow* points must arrive in the stream before a point is declared as an outlier/change point.
*minErrorThreshold**	the minimum error necessary to trigger outlier testing.
*maxErrorThreshold**	the minimum error threshold above which a point is declared as outlier right away.
*minChangeDistance**	the minimum distance between consecutive change points. This parameter controls the granularity of the changes with respect to time.
*windowSize**	the window size for outlier detection. It controls the number of recent points used in deciding whether a newly arrived point is an outlier.
errorThreshold	is the median of the estimated errors of the points within the interval $[t$-*windowSize*,$t]$ that have not been marked as outliers, and it depends on t.
*outlierSensitivity**	is an outlier sensitivity factor in comparing the error estimate for a point with the errorThreshold. It controls the rate of false positives/negatives.
*overshootSensitivity**	is a sensitivity factor for claiming an error overshoot with respect to the errorThreshold.
errorOvershoots	is the number of points within the interval $[t$-*windowSize*,$t]$ that have estimated error larger than *overshootSensitivity* $\times$ *errorThreshold*.
*maxErrorOvershoots**	the maximum number of error overshoots allowed for an outlier (if there too many outliers in the windowSize, then testing for a change point is done).
*changeOutlierWindow**	is the window size considered in testing for changes.
*changeOutlierCount**	is the minimum number of outliers in the among the last *changeOutlierWindow* points in required for declaring a change point.
coefficientGain	is the maximum percentage change of the parameters of $\theta(t)$ with respect to the parameters of $theta(t-1)$.
*gainThreshold**	a threshold for the coefficientGain that forces change detection.
*changeResetWindow**	the lag time in detecting a change. This controls the localization of the change point.

3.1 The Naive Approach

In this approach the RLS algorithm is used to compute the model parameters over all the data seen so far (unrestricted window).

Initially (for the first few points) this approach may be successful in detecting outliers/change points. As more points come in, RLS estimates the model parameters considering all the points seen so far, and thus it gives equal weight to all the points in the unrestricted window. As a result, the changes in the dynamics of the system may be hidden by the dominance of the older points. Figure 3.1 illustrates this phenomenon. As a result RLS by itself is not adequate. The problem is due to the fact that RLS tries to remember all the data points. Instead, if we can make the system forget older points, we should be able to detect outliers/change points better.

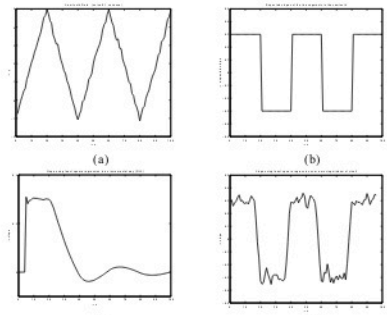


Figure 1. Effect of older points in RLS–based outlier/change point detection. (a) saw–tooth data stream. (b) slopes of the line segments in the saw–tooth. (c) slope estimate using RLS. (d) slope estimate using a moving window of size 5.

3.2 Moving Window Approach

One approach to *forget* older points is to have a moving window over the timeseries/ data stream and estimate the model over this window. At the same time, we would like to maintain the efficiency of the RLS algorithm.

3.2.1 Using Forward–Backward RLS

One method to achieve this would be to derive a backward recursion and alternately use the forward and backward recursions as follows. Suppose the current window spans $[t_1, \ldots, t_n]$. When a new data point comes in at t_{n+1}, we use the forward recursion (equations 7–9) to derive a model for the points in $[t_1, \ldots, t_n, t_{n+1}]$. Then, we use the backward recursion to remove the effects of the point at t_1, and thus get a model for the points in $[t_2, \ldots, t_n, t_{n+1}]$.

In what follows, we derive the backward recursion for RLS. Suppose we have $\hat{\theta}_{old}$ and P_{old} over certain data S, where

$$P_{old} = \left\{ \sum_{k \in S} \varphi(k) \varphi^T(k) \right\}^{-1} \tag{10}$$